A MULTI-STAGE MACHINE LEARNING INTRUSION DETECTION ENGINE FOR KNOWN AND POTENTIALLY NOVEL NETWORK ATTACKS

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

Minor Degree in Machine Learning

Submitted by

Mayank Tiwari (241CV235)
Narra Shanmukha Srida (241CS238)

Under the guidance of

Dr. Suresan Pareth

Department of Information Technology

National Institute of Technology Karnataka, Surathkal

Academic Year: 2026–2027

Certificate

This is to certify that the project entitled **"A Multi-Stage Machine Learning Intrusion Detection Engine for Known and Potentially Novel Network Attacks"** is a bona fide work carried out by Mayank Tiwari (241CV235) and Narra Shanmukha Srida (241CS238) under my supervision and guidance in partial fulfilment of the requirements for the award of the degree of Minor Degree in Machine Learning.

Project Guide

Head of Department

Date: 29/10/2026

Place: NITK Surathkal

Declaration

We hereby declare that the project entitled **"A Multi-Stage Machine Learning Intrusion Detection Engine for Known and Potentially Novel Network Attacks"** is our original work carried out under the guidance of Dr. Suresan Pareth. This work has not been submitted elsewhere for the award of any degree or diploma, except where acknowledged in the report.

Place: NITK Surathkal
Date: 29/10/2026

Signature of Students

Acknowledgement

We express our sincere gratitude to our project guide, Dr. Suresan Pareth, for the guidance and feedback provided throughout this work, and to the Head of the Department, the faculty members and the institution for their support and encouragement. We also thank the Canadian Institute for Cybersecurity, University of New Brunswick, for making the CICIDS2017 dataset publicly available.

Abstract

Signature-based intrusion detection recognises only attacks it has rules for, and a single supervised classifier trained on flow records struggles with severe class imbalance, false alarms and unfamiliar traffic. This project designs, implements and evaluates a **multi-stage machine learning intrusion detection engine** for network flow records. A high-recall approximate RBF support vector machine screens every flow; an out-of-fold stacked ensemble of Random Forest, XGBoost and Logistic Regression names the attack family of screened flows; a benign-only PyTorch autoencoder supplies independent anomaly evidence; and a transparent rule-based fusion step combines the stages while keeping every score. An optional Hidden Markov Model stage for session data is implemented but reported as unavailable, because CICIDS2017 has no session identifiers.

The engine is trained and evaluated on the full CICIDS2017 dataset (2,830,743 flows, 78 features, nine traffic families) with preprocessing fitted on training data only, thresholds and hyperparameters selected on validation data, and a single test evaluation. Under a family-stratified split the engine detects 98.27% of attacks at a 4.31% benign false-positive rate, attributes attack families with 99.95% accuracy, and costs 0.075 ms per flow; validation-based tuning reduced the SVM gate's false-positive rate from 10.8% to 3.3%. Single tree ensembles nevertheless detect attacks with fewer false alarms than the gate, which is the main bottleneck, while the autoencoder catches rare attacks that the gate misses. A second experiment that holds out entire capture files shows that all models degrade sharply on an unseen attack day, so random-split results on CICIDS2017 overstate real-world performance. The engine is served through a command-line predictor, a FastAPI service and a Streamlit dashboard that share one inference implementation.

Keywords: Intrusion Detection, CICIDS2017, Support Vector Machine, Stacked Generalisation, Autoencoder, Data Leakage.

Contents

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

Contents v

List of Figures vii

List of Tables viii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Aim and Objectives . . . 1
- 1.4 Scope of the Project . . . 2
- 1.5 Significance of the Study . . . 2
- 1.6 Research Questions . . . 2

2 Literature Review 3
- 2.1 Introduction . . . 3
- 2.2 Existing Approaches . . . 3
- 2.3 Comparison of Existing Studies . . . 3
- 2.4 Limitations of Existing Approaches . . . 4
- 2.5 Research Gap . . . 4
- 2.6 Proposed Contribution . . . 4

3 Dataset and Methodology 5
- 3.1 Dataset Description . . . 5
- 3.2 Feature Description . . . 5
- 3.3 Exploratory Data Analysis . . . 6
- 3.4 Data Preprocessing . . . 6
- 3.5 Feature Engineering . . . 6
- 3.6 Dataset Splitting . . . 6
- 3.7 Proposed Methodology . . . 7
- 3.8 Experimental Setup . . . 7

4 Model Development 9
- 4.1 Introduction . . . 9
- 4.2 Algorithm Selection . . . 9
- 4.3 Mathematical Foundations . . . 9
- 4.4 Model Architecture or Algorithm Steps . . . 10
- 4.5 Model Training . . . 10

4.6 Hyperparameter Selection 11
4.7 Implementation Details 12

5 Results and Discussion **13**
5.1 Introduction 13
5.2 Evaluation Metrics 13
5.3 Experimental Results 13
5.4 Confusion Matrix 15
5.5 Comparison of Models 16
5.6 Interpretation of Results 17
5.7 Error Analysis 18
5.8 Limitations 19

6 Conclusion and Future Work **20**
6.1 Summary of the Project 20
6.2 Achievement of Objectives 20
6.3 Major Findings 20
6.4 Limitations 21
6.5 Future Work 21

Bibliography **22**

A Supplementary Material **24**
A.1 Additional Experimental Results 24
A.2 Source Code 25
A.3 Additional Dataset Information 25

List of Figures

3.1 Flows per traffic family in CICIDS2017 (log scale) 5
3.2 Workflow (top) and multi-stage inference architecture (bottom) . . 7

5.1 ROC (log-scaled FPR) and precision-recall curves, Experiment 1 test set 14
5.2 Autoencoder reconstruction error for benign and attack test flows (Experiment 1) 15
5.3 Fraction of test flows flagged per attack family by the SVM gate and the autoencoder 15
5.4 Stacked ensemble confusion matrix on attack flows (row-normalised) 16
5.5 Final alert label by true family (row-normalised; $\leq$20,000 test flows per family) 16

List of Tables

2.1 Comparison of representative approaches 4

3.1 CICIDS2017 capture files and their traffic 5
3.2 Experimental environment 8

4.1 Decision fusion rules 10
4.2 SVM gate hyperparameter search on the validation split (first row: original setting; bold: selected) 11
4.3 Final hyperparameters 11

5.1 Binary detection on the Experiment 1 test set (attack = positive class) 13
5.2 Attack-family attribution on true attack flows of the Experiment 1 test set 14
5.3 Per-family results of the stacked ensemble (Experiment 1 test set) . 14
5.4 Experiment 1 (family-stratified rows) vs. Experiment 2 (capture-file split) 17

A.1 Fraction of test flows flagged per family (%); benign row = false-positive rate 24
A.2 Effect of validation-based SVM tuning on the Experiment 1 test set 24
A.3 Ten most important features (Random Forest base model) 24

1. Introduction

1.1 Background and Motivation

Networks face continuous denial-of-service floods, credential brute forcing, web exploits, botnets and scans. Classic network intrusion detection systems (NIDS) match traffic against hand-written signatures, which is precise for known attacks but blind to new behaviour and costly to maintain. Machine learning (ML) offers a complementary approach: flow exporters such as CICFlowMeter summarise each connection into dozens of statistical features, supervised models learn to separate benign from malicious flows, and unsupervised models trained only on normal traffic can flag behaviour unlike anything seen before. Published ML results are, however, often inflated by evaluation shortcuts such as random splits of near-duplicate records or thresholds tuned on test data [8, 7]. This project builds a detection engine from complementary models and evaluates it with a protocol designed to avoid such shortcuts.

1.2 Problem Statement

Given a CICFlowMeter flow record, the system must decide (i) whether the flow is malicious, (ii) if so, which known attack family it belongs to, and (iii) whether it is anomalous relative to normal traffic even if no known family fits. For every flow the output is a final alert label and severity together with the evidence of every stage. A single supervised classifier covers only (i) and (ii), is sensitive to class imbalance (benign traffic is 80.3% of CICIDS2017) and cannot express that traffic is unfamiliar.

1.3 Aim and Objectives

The aim is to develop and rigorously evaluate a multi-stage ML intrusion detection engine for flow records. The objectives are:

- to audit, clean and label CICIDS2017 reproducibly without modifying raw data;
- to build leakage-resistant splits and fit all preprocessing on training data only;
- to implement a high-recall SVM gate, an out-of-fold stacked family classifier,

a benign-only autoencoder and an optional HMM, combined by a transparent fusion policy;

- to compare the engine with single-model baselines using recall, false-positive rate, alert volume and latency; and
- to serve the engine through a CLI, a REST API and a dashboard sharing one implementation.

1.4 Scope of the Project

The system analyses CICFlowMeter flow files offline as a reproducible research prototype for analysts and researchers. Live capture, automatic blocking, online learning and public deployment are out of scope. An anomaly flag means "unusual relative to benign training traffic", not proven zero-day detection.

1.5 Significance of the Study

The project shows how complementary detectors can be combined so that each alert stays explainable, quantifies the detection versus false-alarm trade-off that matters to analysts, and measures how strongly the choice of data split changes reported performance on a widely used benchmark.

1.6 Research Questions

RQ1. Can an approximate RBF SVM act as a high-recall gate with an acceptable false-positive rate at CICIDS2017 scale?

RQ2. Does out-of-fold stacking improve attack-family attribution over its base models?

RQ3. Does a benign-only autoencoder add useful independent evidence?

RQ4. How does the fused engine compare with single-model baselines?

RQ5. How much does performance change on a capture file whose attacks were not seen in training?

2. Literature Review

2.1 Introduction

This chapter reviews intrusion detection datasets, detection methods, ensembles and evaluation pitfalls to position the design choices of the project.

2.2 Existing Approaches

Datasets. CICIDS2017 [1] captured five days of testbed traffic with scripted benign users and scheduled attacks, and extracted flow features with CICFlowMeter. Ring et al. [5] survey intrusion datasets and their differing realism and labelling quality. Engelen et al. [3] and Lanvin et al. [4] found flow-construction and labelling errors in CICIDS2017 whose correction changes reported performance.

Supervised detection. Buczak and Guven [6] survey decision trees, SVMs, ensembles and neural networks for intrusion detection. Random Forest [12] and gradient boosting (XGBoost [13]) are strong on tabular flow features. Kernel SVMs [10] scale poorly with data size, which random Fourier features [11] address by approximating the RBF kernel with an explicit feature map.

Ensembles and anomaly detection. Stacked generalisation [14] trains a meta-learner on base-learner predictions, which must come from data the base learners did not see. Autoencoders trained on normal data use reconstruction error as an anomaly score [15]; Kitsune [16] applies an autoencoder ensemble to online intrusion detection. Hidden Markov Models [17] model sequences of hidden states such as attack stages.

Evaluation pitfalls. Sommer and Paxson [7] argue that ML intrusion detection is harder than other ML tasks because errors are costly and benign traffic is diverse. Arp et al. [8] catalogue pitfalls such as sampling bias and data snooping, and Kaufman et al. [9] formalise data leakage.

2.3 Comparison of Existing Studies

Table 2.1 compares representative works by method and relevance to this project.

Table 2.1: Comparison of representative approaches

Work	Method / data	Relevance and limitation
Sharafaldin et al. [1]	CICIDS2017 creation	Benchmark and flow features used here.
Engelen et al. [3]	Audit of CICIDS2017	Labelling and flow errors; motivates auditing and cautious claims.
Mirsky et al. [16]	Autoencoder ensemble	Unsupervised and fast, but no attack-family attribution.
Wolpert [14]	Stacked generalisation	Combines heterogeneous learners; needs out-of-fold meta-features.
Rahimi & Recht [11]	Random Fourier features	Scales RBF SVMs to millions of rows; quality depends on feature count.
Arp et al. [8]	Pitfalls in security ML	Leakage and sampling bias inflate results; motivates our protocol.

2.4 Limitations of Existing Approaches

Many studies report near-perfect accuracy on CICIDS2017 with a single classifier and a random row split. Because the data contains many exact duplicates and attacks are concentrated in particular capture files, such splits put near-identical flows on both sides and overstate performance. Accuracy is often the headline metric despite extreme imbalance, and single classifiers give no separate evidence for unfamiliar traffic.

2.5 Research Gap

A system is needed that separates screening, attribution and anomaly evidence so that each alert is explainable, is evaluated with leakage-aware splits and operational metrics, and states explicitly when a component (such as temporal modelling) cannot be supported by the data.

2.6 Proposed Contribution

This project contributes a four-stage engine with transparent fusion; a reproducible pipeline with immutable raw data, train-only preprocessing, validation-only tuning, out-of-fold stacking and a single test evaluation; a comparison of a family-stratified split with a capture-file split on the full dataset; and one validated inference engine shared by CLI, REST API and dashboard.

3. Dataset and Methodology

3.1 Dataset Description

The project uses the CICIDS2017 *MachineLearningCVE* release [1, 2]: eight CSV files covering five working days (two files for Thursday, three for Friday) with 2,830,743 labelled flows, 78 numeric features and one label column. Table 3.1 lists the files and their traffic; Figure 3.1 shows the strong imbalance. Benign traffic dominates (80.3%), while Heartbleed (11 flows) and Infiltration (36) are extremely rare.

Table 3.1: CICIDS2017 capture files and their traffic

File	Flows	Benign	Attack families (flows)
Fri PM (DDoS)	225,745	97,718	DDoS (128,027)
Fri PM (PortScan)	286,467	127,537	PortScan (158,930)
Fri AM	191,033	189,067	Bot (1,966)
Mon	529,918	529,918	none
Thu PM (Infiltration)	288,602	288,566	Infiltration (36)
Thu AM (WebAttacks)	170,366	168,186	WebAttack (2,180)
Tue	445,909	432,074	Patator (13,835)
Wed	692,703	440,031	DoS (252,661), Heartbleed (11)

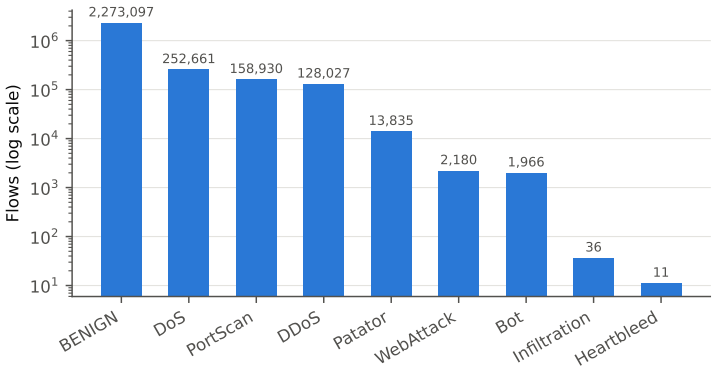


Figure 3.1: Flows per traffic family in CICIDS2017 (log scale)

3.2 Feature Description

Each record summarises one bidirectional flow: volume (packets and bytes per direction), packet-length statistics, timing (duration, inter-arrival, active and idle times), rates (bytes/s, packets/s), TCP flag counts, header and window fields, bulk and subflow statistics, and the destination port. The 15 raw labels are mapped

by a versioned file to a binary target and a nine-class family target (BENIGN, Bot, DDoS, DoS, Heartbleed, Infiltration, Patator, PortScan, WebAttack); the raw label is kept for traceability.

3.3 Exploratory Data Analysis

The audit found (i) extreme imbalance spanning five orders of magnitude; (ii) 256,684 exact duplicate flows (9.1%); (iii) 4,376 infinite values in `Flow Bytes/s` and `Flow Packets/s` and further missing values; (iv) eight columns that are zero in every file; (v) strong file structure: each attack family occurs in only one or two capture files and Monday is purely benign (Table 3.1); and (vi) web-attack labels whose dash is corrupted to the Unicode replacement character in the official archive. Finding (v) drives the splitting strategy.

3.4 Data Preprocessing

Raw files are never modified; their SHA-256 checksums and schemas are recorded. Feature columns are coerced to numbers and infinities replaced by missing values, which are imputed with training-split medians; no rows are dropped. Duplicates are flagged rather than deleted so that their effect can be measured. No categorical encoding is needed, and provenance columns (file, row, flow id) are excluded from the features. Features are standardised with statistics from the training split only and stored as 32-bit floats, which halves memory so the full dataset fits on a 16 GB laptop.

3.5 Feature Engineering

No hand-crafted features are added. The fitted imputer and scaler form a frozen feature contract stored with the deployment manifest, and inference rejects inputs with missing, unexpected or label columns. The HMM uses three emissions (duration, packets/s, mean packet length) when session data exists.

3.6 Dataset Splitting

Two 70/15/15 splits (seed 42) are evaluated:

1. **Experiment 1, family-stratified rows:** rows are assigned at random, stratified on the family label, so every family appears in every partition

(1,981,516 / 424,608 / 424,619 flows). This measures closed-set performance but lets near-identical flows fall on both sides.

2. **Experiment 2, capture-file split:** whole files are assigned to partitions, reshuffling deterministically until each partition contains benign and attack traffic. Training uses Fri PM (PortScan), Fri AM, Mon, Thu PM (Infiltration), Tue, Wed; validation Thu AM (WebAttacks); test Fri PM (DDoS) (225,745 flows).

Validation data is used for the SVM threshold and hyperparameters and for autoencoder early stopping and thresholding; the test split is used once.

3.7 Proposed Methodology

Figure 3.2 shows the workflow and inference architecture. Each pipeline stage is an independently rerunnable script, and one command runs inspection, cleaning, splitting, preprocessing, training of every stage, fusion-policy export, manifest writing and the frozen test evaluation.

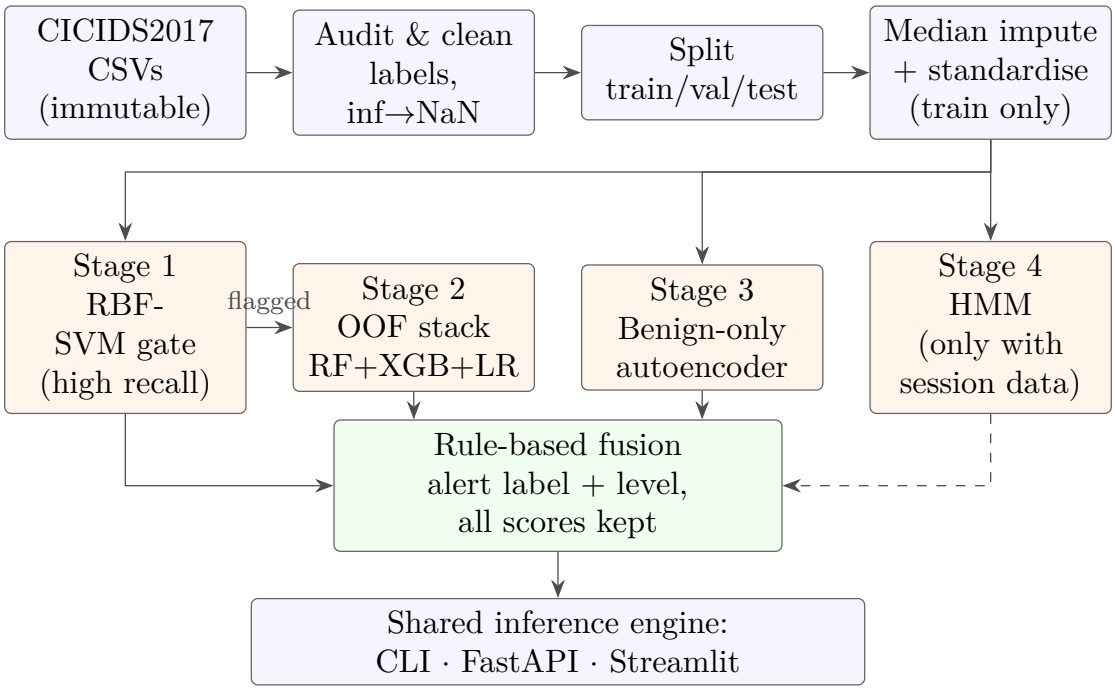


Figure 3.2: Workflow (top) and multi-stage inference architecture (bottom)

3.8 Experimental Setup

Table 3.2: Experimental environment

Item	**Value**
Hardware	16-thread laptop CPU, 16 GB RAM (all models trained on CPU)
Software	Python 3.13, scikit-learn, XGBoost, PyTorch 2 (CPU), hmmlearn, pandas, PyArrow, FastAPI, Streamlit
Reproducibility	seed 42, versioned YAML configs, SHA-256 data manifest, 30 automated tests
Full pipeline run	4 min 9 s for Experiment 1 (peak memory 6.5 GB)

4. Model Development

4.1 Introduction

The engine has four model stages and a fusion step. Each stage answers a different question, so their errors are partly independent, and each keeps its own score so that analysts can see why an alert was raised.

4.2 Algorithm Selection

Stage 1, approximate RBF SVM (binary gate): a maximum-margin classifier with a non-linear kernel; the kernel is approximated so that training scales to millions of flows, and the threshold is set for high recall so later stages see almost every attack. **Stage 2, stacked ensemble (family):** Random Forest (bagging), XGBoost (boosting) and Logistic Regression (linear) make different errors, and a Logistic Regression meta-learner combines their probabilities. **Stage 3, autoencoder (anomaly):** trained on benign flows only, so it needs no attack labels. **Stage 4, Gaussian HMM (temporal):** models the hidden state of a session when ordered session data exists.

4.3 Mathematical Foundations

SVM with random Fourier features. The RBF kernel $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ is approximated by $z(\mathbf{x})^\top z(\mathbf{x}')$, where

$$z(\mathbf{x}) = \sqrt{2/D}\,\Big[\cos(\boldsymbol{\omega}_j^\top \mathbf{x} + b_j)\Big]_{j=1}^{D}, \qquad \boldsymbol{\omega}_j \sim \mathcal{N}(\mathbf{0}, 2\gamma I),\ b_j \sim U[0, 2\pi] \tag{4.1}$$

for D random features [11]. A linear SVM on $z(\mathbf{x})$ minimises the class-weighted regularised hinge loss by stochastic gradient descent:

$$\min_{\mathbf{w},b}\ \frac{\alpha}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} c_{y_i} \max\Big(0,\ 1 - y_i(\mathbf{w}^\top z(\mathbf{x}_i) + b)\Big), \tag{4.2}$$

with class weights c_y inversely proportional to class frequency. A flow is flagged when $s(\mathbf{x}) = \mathbf{w}^\top z(\mathbf{x}) + b \geq \tau$, where τ is the $\lceil 0.98P \rceil$-th largest score among the P validation attacks, the largest threshold giving 98% validation recall.

Out-of-fold stacking. With $K = 5$ stratified folds, each base model m predicts class probabilities $\mathbf{p}_m(\mathbf{x}_i)$ for the fold it was not trained on. The meta-features $\mathbf{q}_i = [\mathbf{p}_{\mathrm{RF}}; \mathbf{p}_{\mathrm{XGB}}; \mathbf{p}_{\mathrm{LR}}]$ train a multinomial logistic regression $P(y = k \mid$

$\mathbf{q}) = \mathrm{softmax}(\mathbf{V}\mathbf{q} + \mathbf{c})_k$; base models are then refitted on all rows. The prediction is $\arg\max_k P(y = k \mid \mathbf{q})$ with confidence $\max_k P(y = k \mid \mathbf{q})$.

Autoencoder. Encoder f and decoder g minimise the mean squared reconstruction error on benign flows. The anomaly score is $a(\mathbf{x}) = \frac{1}{d}\|\mathbf{x} - g(f(\mathbf{x}))\|_2^2$, thresholded at the 99th percentile of benign validation scores (target benign false-positive rate 1%).

HMM. The forward algorithm gives the filtered state posterior $\boldsymbol{\pi}_t = P(h_t \mid \mathbf{o}_{1:t})$ from past observations only, and the next-state distribution is $\boldsymbol{\pi}_t^\top A$ for transition matrix A [17].

Fusion. Table 4.1 lists the rules; the stack is confident when its probability is at least 0.60.

Table 4.1: Decision fusion rules

SVM	Conf. ≥ 0.6	AE	Final label	Level
yes	yes	yes	known_attack_suspected	high
yes	yes	no	known_attack_suspected	medium
no	–	yes	anomaly_review	medium
yes	no	any	suspicious_unclassified	low
no	–	no	benign	none

4.4 Model Architecture or Algorithm Steps

The autoencoder maps 78 inputs through layers of 128, 64 and 16 (bottleneck) units and back through 64 and 128 to 78 outputs, with ReLU hidden activations and a linear output. For a batch of flows, inference (i) validates the schema, rejecting label, unknown and duplicate columns and infinities, then imputes and standardises; (ii) scores every flow with the SVM gate; (iii) runs the stack only on gated flows; (iv) scores every flow with the autoencoder; (v) adds HMM context when session identifiers exist; and (vi) applies the fusion rules, returning every stage's output and the model version.

4.5 Model Training

All models use the 78 standardised features. The SVM gate is trained on a class-stratified sample of the training split with balanced class weights, and its threshold is selected on the validation split. The stack is trained on attack rows only (at most 100,000, sampled per family so that every family keeps at least two rows) with the family target. The autoencoder is trained on up to 100,000 benign flows

with Adam (learning rate 10^{-3}, batch 256) for at most 100 epochs, with early stopping (patience 10) on up to 50,000 benign validation flows.

4.6 Hyperparameter Selection

The SVM gate was tuned by grid search on the validation split only, minimising the benign false-positive rate at the threshold that gives 98% validation recall. Round 1 varied the number of random features, α and the number of epochs; round 2 added the kernel width γ and the training-sample size (Table 4.2). The original setting (one SGD pass, $\alpha = 10^{-4}$, $D = 512$, 100k rows) gave a validation false-positive rate of 10.8%; the selected setting gives 3.3%. Other models use common defaults (Table 4.3).

Table 4.2: SVM gate hyperparameter search on the validation split (first row: original setting; bold: selected)

D	α	**Epochs**	γ	**Rows**	**Val. FPR (%)**	**Val. AUC**	**Fit (s)**
512	10^{-4}	1	scale	100k	10.79	0.9873	0.3
2048	10^{-6}	**5**	**0.2**	**500k**	**3.29**	**0.9945**	**19.9**
2048	10^{-6}	5	0.05	500k	3.41	0.9948	28.3
2048	10^{-5}	5	0.2	500k	3.72	0.9932	20.2
512	10^{-6}	5	scale	500k	3.95	0.9920	5.1
2048	10^{-5}	5	0.05	500k	3.98	0.9915	20.0
512	10^{-6}	5	0.05	500k	4.08	0.9930	5.0

FPR measured at the threshold giving 98% validation recall. 36 configurations were evaluated in two rounds; the six best of round 2 are shown.

Table 4.3: Final hyperparameters

Model	**Setting**
SVM gate	$D = 2048$ random features, $\gamma = 0.2$, $\alpha = 10^{-6}$, 5 SGD epochs, 500,000 training rows; balanced class weights; recall target 0.98
Random Forest	200 trees, balanced class weights
XGBoost	200 trees, depth 6, learning rate 0.08, subsample and column sample 0.9
Logistic Regression	base and meta learner, balanced class weights
Stacking	5 stratified folds, $\leq$100k rows, confidence threshold 0.60
Autoencoder	128-64-16, Adam 10^{-3}, batch 256, $\leq$100 epochs, patience 10
HMM	Gaussian, diagonal covariance, 2 states, 3 emission features

4.7 Implementation Details

The system is a Python package (`src/ids`) plus numbered pipeline scripts. A single `InferenceEngine` is shared by the CLI predictor, the FastAPI service (`/health`, `/model-info`, `/predict`, `/predict-batch`, `/predict-file`; at most 10,000 rows or 10 MB per request) and the Streamlit dashboard. Training writes a versioned manifest of features, labels and thresholds, and loading fails if the saved models disagree with it.

5. Results and Discussion

5.1 Introduction

Both experiments use identical models and hyperparameters and differ only in the split. Experiment 1 is the main closed-set evaluation; Experiment 2 tests generalisation to an unseen capture file. Baselines (Logistic Regression, Random Forest, XGBoost) are trained on the same budget as the SVM gate's original setting (100,000 family-stratified rows) with a 0.5 probability threshold. All results are on test data.

5.2 Evaluation Metrics

With attacks as the positive class: recall (detection rate) = $\frac{TP}{TP+FN}$; precision = $\frac{TP}{TP+FP}$; $F_1 = \frac{2PR}{P+R}$; false-positive rate (FPR) = $\frac{FP}{FP+TN}$, also given as alerts per 1000 benign flows (analyst workload); ROC-AUC summarises ranking quality. Family attribution uses macro-F1, so that rare families count equally. Accuracy is not a headline metric: labelling everything benign would score 80.3%. Latency is the complete engine's time per flow.

5.3 Experimental Results

Table 5.1 and Figure 5.1 compare binary detection in Experiment 1.

Table 5.1: Binary detection on the Experiment 1 test set (attack = positive class)

Model	Prec. %	Rec. %	F1 %	FPR %	Alerts/1k	ROC-AUC	ms/flow
Logistic regression	71.16	95.39	81.51	9.487	94.9	0.9782	0.0001
Random forest	99.56	99.69	99.62	0.109	1.1	0.9999	0.0007
XGBoost	99.65	99.83	99.74	0.085	0.9	0.9999	0.0002
SVM gate (validation-tuned)	87.98	98.03	92.73	3.285	32.9	0.9946	0.0066
Full fused system	84.82	98.27	91.05	4.314	43.1	–	0.0749

Baselines use a 0.5 probability threshold; the SVM gate uses its validation-tuned threshold. ms/flow for baselines and the gate is model scoring only; for the fused system it is the complete engine including validation and fusion.

The fused engine detects 98.27% of attacks with 43.1 alerts per 1000 benign flows at 0.075 ms per flow (about 13,347 flows/s in one CPU process). Table 5.2 compares family attribution on true attack flows, and Table 5.3 gives per-family results of the stack.

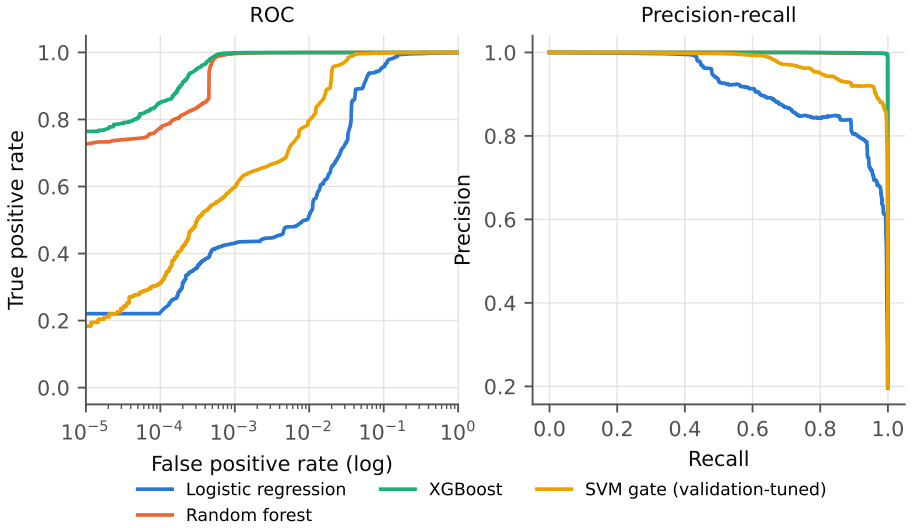


Figure 5.1: ROC (log-scaled FPR) and precision-recall curves, Experiment 1 test set

Table 5.2: Attack-family attribution on true attack flows of the Experiment 1 test set

Model	Accuracy %	Macro-F1	Weighted-F1	Bal. acc. %
Random forest	99.96	0.985	1.000	97.60
XGBoost	99.97	0.961	1.000	93.47
Logistic regression	99.57	0.869	0.996	90.52
Stacked ensemble (OOF meta)	99.95	0.922	0.999	97.70

Base models are the stack's own base learners refitted on its 8-family training sample. The end-to-end cascade (gate misses counted as errors) has macro-F1 0.614.

Table 5.3: Per-family results of the stacked ensemble (Experiment 1 test set)

Family	Precision %	Recall %	F1 %	Test flows
DoS	99.97	99.95	99.96	37,900
PortScan	99.99	99.93	99.96	23,840
DDoS	99.99	99.98	99.98	19,205
Patator	100.00	100.00	100.00	2,076
WebAttack	94.44	98.78	96.56	327
Bot	100.00	99.66	99.83	296
Infiltration	27.78	83.33	41.67	6
Heartbleed	100.00	100.00	100.00	3

The autoencoder, trained without attack labels, reaches a ROC-AUC of 0.811 and flags 40.5% of attacks at a 1.13% benign false-positive rate (target 1%). Figures 5.2 and 5.3 show its score separation and which families each detector catches.

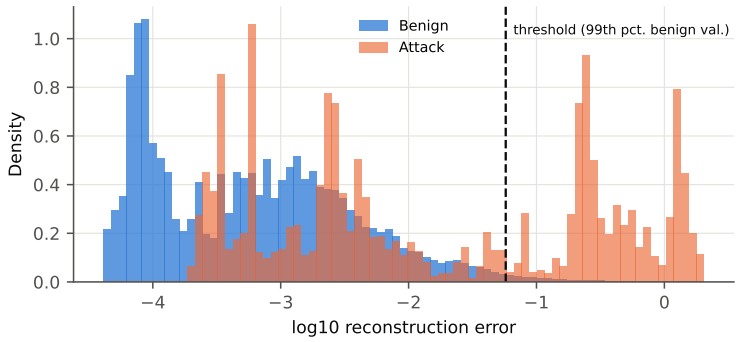


Figure 5.2: Autoencoder reconstruction error for benign and attack test flows (Experiment 1)

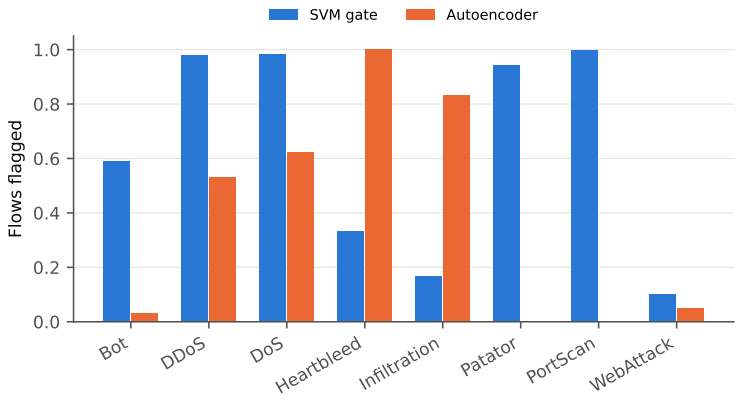


Figure 5.3: Fraction of test flows flagged per attack family by the SVM gate and the autoencoder

5.4 Confusion Matrix

Figure 5.4 shows the stack's row-normalised confusion matrix on attack flows and Figure 5.5 the engine's final alert labels per true family.

Given a true attack flow, the stacked ensemble assigns the correct family to 99.95% of flows (Figure 5.4, Table 5.3). Every family except Infiltration is recognised with recall of at least 98.8%. The weak point is Infiltration, with only six test flows. Five of the six are identified correctly (recall 83%), but 13 flows from other families are also labelled Infiltration, so its precision is only 28% and its F1 is 0.42. The balanced class weights give this 36-flow class a very large weight, which pushes borderline flows towards it.

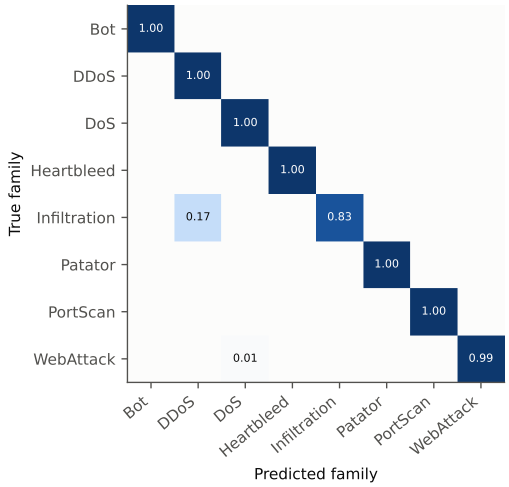


Figure 5.4: Stacked ensemble confusion matrix on attack flows (row-normalised)

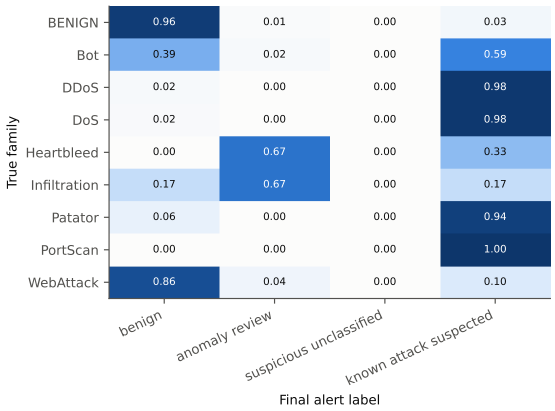


Figure 5.5: Final alert label by true family (row-normalised; ≤20,000 test flows per family)

Figure 5.5 shows the end-to-end picture. Flows from the large families are almost all labelled `known_attack_suspected`. Most Heartbleed and Infiltration flows receive `anomaly_review`: the SVM gate misses them, but the autoencoder flags them as unusual. Most WebAttack flows end as `benign`, because both the gate and the autoencoder miss them.

5.5 Comparison of Models

Binary detection. On the Experiment 1 test set the single tree ensembles are the strongest binary detectors (Table 5.1): XGBoost reaches 99.83% recall at 0.085% FPR (0.9 alerts per 1000 benign flows), and Random Forest 99.69% at 0.109%. The tuned SVM gate reaches its designed recall (98.03%) but at a higher false-positive rate (3.29%, ROC-AUC 0.9946), and Logistic Regression is the weakest (9.487% FPR). The fused system inherits the gate's recall and adds the autoencoder's anomaly alerts, giving 98.27% recall at 4.31% FPR. The multi-stage design therefore does not beat a well-configured tree ensemble on raw binary accuracy in this closed-set setting. Its value is in attribution, independent anomaly evidence and explainability, and its binary performance is limited by the gate.

Family attribution. All four models attribute families of true attack flows with more than 99.5% accuracy (Table 5.2). On macro-F1, which weights the six-flow Infiltration class equally with DoS, the Random Forest base learner (0.985) scores higher than the stacked ensemble (0.922), XGBoost (0.961) and Logistic Regression (0.869). Stacking therefore did not improve over its best base model here. Its meta-learner is trained with balanced class weights on out-of-fold probabilities,

and with only a few dozen Infiltration and Heartbleed rows the meta-learner's estimates for these classes are noisy.

Generalisation to an unseen capture. In Experiment 2 the test file is the Friday afternoon DDoS capture, and DDoS never appears in training. The trained models behave very differently there (Table 5.4). The tree ensembles keep their precision but detect only 63.50% (Random Forest) and 62.59% (XGBoost) of the unseen DDoS flows. The SVM gate's threshold was calibrated on the only validation file, which contains web attacks. On the new day it flags almost all traffic: 99.94% of attacks but also 96.98% of benign flows. The autoencoder's benign false-positive rate rises from 1.13% to 17.06%, which shows that benign traffic itself shifts between capture days. When the family classifier is given true DDoS flows, it labels them as the closest known families: about 84% as DoS and 16% as PortScan (from the Experiment 2 confusion matrix).

Table 5.4: Experiment 1 (family-stratified rows) vs. Experiment 2 (capture-file split)

Metric	**Experiment 1**	**Experiment 2**
Test flows	424,619	225,745
Attack families in test	Bot, DDoS, DoS, Heartbleed, Infiltration, Patator, PortScan, WebAttack	DDoS
Families unseen in training	none	DDoS
Test flows with exact copy in train	13.8%	3.3%
SVM gate recall / FPR	98.03 / 3.29	99.94 / 96.98
Autoencoder attack TPR / benign FPR	40.46 / 1.13	51.32 / 17.06
Fused recall / FPR	98.27 / 4.31	100.00 / 97.29
Fused alerts per 1000 benign	43.1	972.9
Stack accuracy on attack flows	99.95%	0.00%

5.6 Interpretation of Results

Why tree ensembles dominate. CICFlowMeter features have heterogeneous scales and heavy tails, and many attacks are separated by sharp thresholds on a few features (destination port, initial window size, header lengths and packet-length maxima; Table A.3). Axis-aligned tree splits capture such rules directly. The approximate RBF SVM must express the same boundaries through a smooth kernel on standardised features, using 2048 random features and a 500k-row sam-

ple. Validation-based tuning cut its false-positive rate from 10.8% to 3.3%, but it remains well behind the trees.

Why the split matters so much. 9.1% of all flows are exact duplicates. Under the family-stratified split, 13.8% of test flows have an identical feature vector in the training set, compared with 3.3% under the capture-file split. Closed-set results are therefore partly a measure of recognition, not generalisation. The large drop in Experiment 2 matches the "closed world" concern of Sommer and Paxson [7]: models learn the attacks and the benign traffic of the days they saw.

Value of the anomaly stage. The autoencoder alone is a weak detector (ROC-AUC 0.811, 40.5% of attacks flagged at 1.13% benign FPR), but it is complementary. It flags exactly the rare families the gate misses, and these become `anomaly_review` alerts instead of silent misses. The anomaly stage contributes coverage of unusual traffic, not overall accuracy.

Feature importance and shortcuts. The destination port is the single most important Random Forest feature. In CICIDS2017 several attacks target fixed ports (for example FTP and SSH brute forcing, and web attacks on port 80), so this feature can act as a dataset-specific shortcut that would not transfer to networks with different services.

5.7 Error Analysis

- **Web attacks are mostly missed.** Only about 10% of WebAttack test flows pass the SVM gate, and the autoencoder flags few of them, so most are labelled benign (Figure 5.3). Brute-force and XSS requests are ordinary-looking HTTP flows to port 80 whose statistics overlap heavily with benign web browsing. The family classifier separates them well once they are gated (F1 above 96%), so the gate is the bottleneck.
- **Bot traffic is partly missed.** About 59% of Bot flows are detected; the missed ones are likely low-volume periodic connections whose statistics resemble benign traffic.
- **Rare families rely on the anomaly stage.** The gate detects only one of three Heartbleed flows and one of six Infiltration flows, but the autoencoder flags most of them. With so few examples, per-family estimates for these classes have very high variance.
- **False positives.** 43.1 of every 1000 benign flows raise an alert. Most come from the gate (32.9 per 1000) and the rest from anomaly alerts on unusual but benign flows (the autoencoder's 1.13% design rate).

- **Cascade metric.** When gate misses are counted as a separate wrong class, the end-to-end family macro-F1 is 0.614. This value is driven mainly by the missed WebAttack, Bot and rare-family flows above, not by attribution errors.

5.8 Limitations

CICIDS2017 is a testbed capture with known labelling errors [3, 4], and Heartbleed and Infiltration are too rare for reliable per-family estimates. Closed-set results include near-duplicates across partitions. The SVM gate is an approximation trained on a 500k-row sample, and the stack and autoencoder use 100k-row samples. The destination port may act as a shortcut feature, the HMM could not be evaluated without session identifiers, and the fusion thresholds are fixed rules rather than optimised values.

6. Conclusion and Future Work

6.1 Summary of the Project

A reproducible multi-stage intrusion detection engine was built for network flow records and evaluated on the full CICIDS2017 dataset. Raw data is audited without modification, preprocessing is fitted on training data, thresholds and hyperparameters are selected on validation data, and test data is used once. An approximate RBF SVM gate, an out-of-fold stacked family classifier, a benign-only autoencoder and an optional HMM are combined through transparent fusion and served by a CLI, a REST API and a dashboard sharing one inference implementation.

6.2 Achievement of Objectives

All objectives were met: the dataset was audited (duplicates, non-finite values, constant columns, corrupted labels) and labelled reproducibly; two leakage-aware splits were implemented; all stages and the fusion step were implemented and covered by 30 automated tests; the engine was compared with baselines on recall, false-positive rate, alert volume and latency; and the CLI and API outputs were verified to agree.

6.3 Major Findings

1. **Gate (RQ1):** an approximate RBF SVM can serve as a 98%-recall gate at full CICIDS2017 scale. Its false-positive rate depends strongly on hyperparameters: validation-based tuning reduced it from 10.8% to 3.3% (test: 3.29%). It remains the main source of false alarms and missed web attacks.
2. **Stacking (RQ2):** family attribution is above 99.5% accurate for all models, but out-of-fold stacking did not improve macro-F1 over its best base learner, the Random Forest (0.922 vs. 0.985), because of very rare families.
3. **Autoencoder (RQ3):** the benign-only autoencoder is weak alone but complementary. It turns most rare-family flows missed by the gate (Heartbleed, Infiltration) into `anomaly_review` alerts.
4. **Baselines (RQ4):** single tree ensembles (XGBoost: 99.83% recall at 0.085% FPR) beat the fused engine on binary detection in the closed-set

setting. The engine's advantages are explainable, staged evidence and family attribution, at 0.075 ms per flow.

5. **Generalisation (RQ5):** on an unseen capture day with an unseen attack family, every model degrades. Tree ensembles miss over a third of the DDoS flows, the gate's validation-calibrated threshold flags most benign traffic, and benign traffic itself shifts. Results from random splits of CICIDS2017 (where 13.8% of test flows have an exact copy in training) substantially overstate real-world performance.

6.4 Limitations

The main limitations are the testbed nature and label errors of CICIDS2017, the dependence of results on the split, bounded training samples, the possible destination-port shortcut and the lack of session data for the temporal stage.

6.5 Future Work

- Replace or complement the SVM gate with a tree-based gate, and tune the fusion thresholds on validation data against an explicit alert budget.
- Evaluate on the corrected CICIDS2017 release [3] and other datasets, and run withheld-family experiments for every family.
- Replace the destination port with protocol-level features, reconstruct sessions so the HMM can be evaluated, and connect a live CICFlowMeter feed to the API.

Bibliography

[1] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] Canadian Institute for Cybersecurity, "Intrusion Detection Evaluation Dataset (CIC-IDS2017)," University of New Brunswick. URL: https://www.unb.ca/cic/datasets/ids-2017.html, Accessed: 28 October 2026.

[3] G. Engelen, V. Rimmer and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[4] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé and E. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Proc. 17th Int. Conf. Risks and Security of Internet and Systems (CRiSIS)*, LNCS, Springer, 2022, pp. 18–33.

[5] M. Ring, S. Wunderlich, D. Scheuring, D. Landes and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[6] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[7] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305–316.

[8] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022.

[9] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, 2012.

[10] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[11] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems (NIPS)*, 2007.

[12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[14] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[15] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," in *Proc. 2nd Workshop on Machine Learning for Sensory Data Analysis (MLSDA)*, 2014.

[16] Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

[17] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[18] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[19] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

A. Supplementary Material

A.1 Additional Experimental Results

Table A.1: Fraction of test flows flagged per family (%); benign row = false-positive rate

Family	E1 gate	E1 AE	E2 gate	E2 AE
BENIGN	3.3	1.1	97.0	17.1
Bot	59.1	3.0	–	–
DDoS	98.1	53.1	99.9	51.3
DoS	98.1	62.2	–	–
Heartbleed	33.3	100.0	–	–
Infiltration	16.7	83.3	–	–
Patator	94.1	0.0	–	–
PortScan	99.9	0.2	–	–
WebAttack	10.1	4.9	–	–

Table A.2: Effect of validation-based SVM tuning on the Experiment 1 test set

Metric	Before tuning	After tuning
SVM gate recall (%)	97.93	98.03
SVM gate FPR (%)	10.78	3.29
Fused recall (%)	98.17	98.27
Fused alerts per 1000 benign	115.9	43.1

Table A.3: Ten most important features (Random Forest base model)

Feature	Importance
Destination Port	0.0834
Init_Win_bytes_backward	0.0593
Fwd Header Length.1	0.0445
Fwd Packet Length Max	0.0409
Fwd Header Length	0.0375
min_seg_size_forward	0.0310
Subflow Fwd Bytes	0.0298
Total Length of Bwd Packets	0.0294
Average Packet Size	0.0288
Bwd Header Length	0.0288

A.2 Source Code

The code, configurations and tests are in the project repository (https://github.com/srida01/ml_project). `python scripts/run_pipeline.py --skip-baselines` reproduces Experiment 2, and adding the option

`--split-config configs/split_row_stratified.yaml`

reproduces Experiment 1. The hyperparameter search is reproduced by `scripts/13_tune_svm.py`; every table and figure is produced by `scripts/12_report_analysis.py` together with `scripts/14_split_overlap.py`. The implementation uses scikit-learn [18] and PyTorch [19].

A.3 Additional Dataset Information

The eight columns that are zero in every file are `Bwd PSH Flags`, `Bwd URG Flags` and the six bulk-rate features (forward and backward *Avg Bytes/Bulk*, *Avg Packets/Bulk* and *Avg Bulk Rate*). Label mapping: DoS Hulk, GoldenEye, slowloris and Slowhttptest → DoS; FTP- and SSH-Patator → Patator; Web Attack Brute Force, XSS and Sql Injection → WebAttack; the other labels keep their names. Archive SHA-256 prefix: `c3f26274b36c837c`.